Name(s): Yasaman Saatsaz, Gilbert Teng

Roles / Contributions:

- **Yasaman:**
    - Ship generation function and relevant helper functions
        - generate_ship
        - get_neighbors
        - find_dead_ends
        - print_ship
    - Search algorithm and relevant helper functions
        - reconstruct_path
        - manhattan_distance
        - a_star_search
    - Finding configurations in T with max expected values (for write-up only)
        - analyze_max_T_configurations
    - Machine learning algorithm
        - class RatNet
            - a constructor for a neural network model
        - build_dataset_from_T
            - creates a dataset from array T
        - get_batch
            - gets a random sample of the data to train with
        - train_model
            - a function that trains the model using the random sample
        - evaluate_model
            - a function that evaluates the model on testing data
    - Graphing functions (for write-up only)
        - train_and_track
            - similar to our train_model function, but also tracks both the training and testing losses (for graphing)
        - plot_training_loss
            - only graphs the average training loss over time
        - plot_train_vs_test_loss
            - graphs both the training and testing losses
- **Gilbert:**
    - T array generation function and relevant helper functions
        - compute_value_function
        - best_bot_move

        - rat_movement
        - rat_search
    - Comparing shortest path algorithm with optimal path algorithm
        - shortest_path_vs_optimal_path
    - Augmented neural network and helper functions
        - estimate_blocked_cells_between
            - calculates the number of blocked cells between the bot and the rat
        - build_augmented_dataset_from_T
            - builds a training dataset that includes bot/rat positions and a new feature: blocked ratio between bot/rat positions
        - class RatNetAugmented
            - the defined class for a new augmented neural network that has an increase in the number of inputs
        - get_augmented_batch
            - gets a random batch from the augmented dataset
        - train_augmented_model
            - trains the augmented neural network model
        - test_on_new_ship
            - tests the model on a new ship
        - test_on_new_ship_og
            - tests the model on the original ship
        - main_augmented_training_and_generalization
            - trains and tests the augmented model on multiple ships
- **Both:**
    - Main function for testing

Project 3 - Machine Learning + MDPs

Write-Up / Report – Rat Search Predictions

Let's take into account a ship that can be represented as a D x D grid, where D is some number (like 30). Some cells on the ship are blocked, while some are open. There is a rat on the ship as well as a bot. The bot's job is to traverse the ship and find the rat in as few moves as possible. The bot has a map of the ship, so it knows which cells are blocked and which are open. The bot and the rat alternate moves, and the rat moves to a random neighboring open cell during its turn. When the bot moves into the same cell that the rat is in, the bot has successfully captured the rat. The bot's goal is to find the rat in as few moves as possible.

Let T (bx, by , rx, ry) be the minimal expected number of moves to remove the rat, when the bot starts at cell (bx, by) in the ship, and the rat starts at cell (rx, ry). Essentially, T is a D x D x D x D array of (bot, rat) configurations in the ship, where the value stored for each (bot, rat) tuple is the expected number of steps for the bot to reach the rat for that specific configuration. We can create the configuration array T by using value iteration.

Generally, value iteration is the process of evaluating the smallest cost of moving to a certain state by considering the cost of moving to different states and the utility of possible actions (we're assuming that we have an MDP that maps our possible states and actions). The process usually starts with some random guess, and then improving on that guess. For our purposes, the bot is determining the minimum cost by calculating the cost of moving to different neighboring cells and the utility of which action the bot should take, meaning that the bot would determine its best move as the move that results in the smallest number of expected steps to catch the rat in that configuration.

Let's go through this process step by step.

First, we could initialize T as a D x D x D x D array and have our initial guess be some random number like 500 (the initial guess is usually a bad guess, but we'll improve our guess through value iteration). This means that for every (bot, rat) configuration, the expected number of steps is 500. Of course, we have to consider the fact that the ship has both blocked and open cells. In other words, we have to be careful of blocked cells where the bot and rat

can't be. Due to our project's constraints, combinations such as either the bot or rat in a blocked cell should never happen, but the array would still allocate memory for such situations. To mitigate this, we'll only consider open cells; we can create a list of open cells inside the ship, which we can use to find potential bot and rat locations. We also have another failsafe, which takes place after value iteration and will be explained later in this report.

For every possible bot location (i.e. every open cell in the ship), if the bot's coordinates are the same as the rat's, this means that the bot found the rat and the expected number of steps can be set to 0. We can deduce this: **it's easier to compute T for configurations where the bot and the rat are in the same cell** because the expected number of steps that the bot would take to find the rat is 0; this is when (bx, by) = (rx, ry). Some examples of this (assuming the following cells are open and within the ship's dimensions) can be:

- (bx, by) = (0, 0) and (rx, ry) = (0, 0)
- (bx, by) = (4, 3) and (rx, ry) = (4, 3)
- (bx, by) = (2, 6) and (rx, ry) = (2, 6)

We can now start value iteration for other configurations by looping over all valid bot and rat positions (all open cells), though we can skip over instances where the bot and rat's positions are the same since we've just accounted for configurations like those by setting the expected steps to 0. For each of the 4 possible directions the bot can move it for that configuration, we first initialize the minimum expected number of steps to be infinity (we will change this later upon finding a new minimum (i.e. a smaller number of expected steps for that configuration)). Then, we calculate the bot's new position for moving and check to see if the new position is still within the bounds of the ship and an open cell. We can then compute all valid moves that the rat can make from its current position in a similar fashion by calculating its position for each of the 4 directions and checking if it is within bounds and an open cell. If the rat has no valid moves, it will not move, so its position will stay the same. Otherwise, it will pick a random possible position to move to from the list of valid moves it can make.

The expected number of steps is calculated by taking the sum of all the rat's new possible locations (assuming the bot took a step to its new location already) and dividing it by the number of rat moves. Once the bot has calculated movements for all possible directions (and the expected value of the rat's movement has been calculated for each one), we can now find the minimum cost (i.e. the bot's best move). One way of finding the minimum cost is by taking the new cost (the expected value we just calculated) plus 1 to account for the bot's move, and comparing it with the configuration's current cost (the expected value it had before

we computed value iteration; we'll select the smallest of these two values and consider it our best estimate. We can save our new estimate for expected value in our T array for that configuration, effectively replacing our terrible initial guess with a better guess.

This whole process can be modeled with the following formula:

$$T(bx, by, rx, ry) = \min_{(nbx,\ nby)} \left[1 + \frac{1}{|R|} \sum_{(nrx,\ nry) \in R} T(nbx,\ nby,\ nrx,\ nry)\right.$$

Where:

- (nbx, nby) = the new position of the bot after moving in one of the 4 valid directions
- (nrx, nry) = the new position of the rat after moving in one of the 4 valid directions
- R = the set of all valid rat positions
- 1 + → accounts for the step the bot just took

In simpler terms, we are going to choose the bot move with the smallest cost out of all possible bot moves ($\min_{(nbx,nby)}$) after one bot move (1 +), one rat move ($\frac{1}{|R|}$), and the expected number of steps for configurations involving the bot and rat's new positions ($T(nbx,\ nby,\ nrx,\ nry)$). The bot move with the smallest cost is considered to be **the bot's most optimal action**. By computing these calculations, we can determine the expected number of steps for the current configuration (T(bx, by, rx, ry)).

Of course, after calculating the expected number of steps for each configuration in T, we have to check if the values in T converged, meaning that the change in values has been reduced to either 0 or a very small number. We look at every possible configuration in T and calculate the change between the old expected value and our new expected value, then add that change to the total change (for the average). After calculating the change for all configurations in T, we calculate the average change, which is the total change divided by the number of configurations in T (we assume we get the absolute average change).

If the average change is smaller than some threshold of change that we pre-determined, then that means the values converged and have stabilized, so we can stop performing value iteration. Otherwise, we rerun value iteration on our T array of new values. To make things simpler, we can make a copy of T for every time we run value iteration and

initialize it with our new expected values, that way we can easily track the change between our old T array and our new T array. After value iteration is finally complete and our values have converged, we check to see if we still have any values in T that are still too large (such as 400 or infinity); these values represent configurations where the bot can't realistically reach the rat, such as when the bot or rat are on blocked cells, or surrounded by blocked cells. This is our extra failsafe to account for impossible combinations since the ship has both blocked and open cells. This also means that **the bot won't always be able to catch the rat** as well.

Although our project's constraints specified that D should not be any smaller than 30, for simplicity's sake, let's try testing this on a 20 x 20 ship with probability p of blocked cells being 0.5.

Let's assume the picture above is our ship, with blocked cells being marked with "B" and open cells marked with "O". *(Notice that a 30 x 30 ship, while it can be printed, would be harder to interpret for readers of this report)*

If we're looking for max configurations of T, we're looking for configurations where either the bot or rat are in a blocked cell, or the bot is unable to reach the rat (such as instances where there is no open path to the rat or the rat is trapped by blocked cells). The values for such configurations would most likely be some large number such as infinity, as our algorithm for generating T initially set the expected value for each configuration to be infinity, which

would then be updated with better values as value iteration progressed. If the value was never updated from infinity, that means that there were no better moves the bot could take to reach the rat, making it impossible for the bot to find the rat.

Following this logic, we can compute a set of “max-T configurations”:

```
Bot: (19,17)  |  Rat: (3,0)
Bot: (19,17)  |  Rat: (3,2)
Bot: (19,17)  |  Rat: (3,4)
Bot: (19,17)  |  Rat: (3,6)
Bot: (19,17)  |  Rat: (3,8)
Bot: (19,17)  |  Rat: (3,10)
Bot: (19,17)  |  Rat: (3,12)
Bot: (19,17)  |  Rat: (3,14)
Bot: (19,17)  |  Rat: (3,15)
Bot: (19,17)  |  Rat: (3,16)
Bot: (19,17)  |  Rat: (3,17)
Bot: (19,17)  |  Rat: (3,18)
Bot: (19,17)  |  Rat: (4,0)
Bot: (19,17)  |  Rat: (4,1)
Bot: (19,17)  |  Rat: (4,3)
Bot: (19,17)  |  Rat: (4,5)
Bot: (19,17)  |  Rat: (4,6)
Bot: (19,17)  |  Rat: (4,7)
Bot: (19,17)  |  Rat: (4,9)
Bot: (19,17)  |  Rat: (4,10)
```

(Note that this isn’t all of the possible max-T configurations; the list is much longer so we can just look at a smaller sample of configurations)

There are multiple max-T configurations where the bot is at coordinates (19, 17), which is a blocked cell according to our map of the ship. Because the bot is in a blocked cell, it wouldn’t be possible for the bot to ever find the rat, so the expected value of configurations where the bot is at (19, 17) would remain at infinity, making them max-T configurations.

How would our “optimal path” algorithm compare with an algorithm that computes the shortest path, which uses A* search with Manhattan distance as a heuristic? In other words, how often is the bot’s optimal action just to move along the shortest path to the rat (calculated as a percentage of total configurations)?

We can go about making this comparison by looking at each individual move that the bot takes using our optimal path algorithm and compare it with the corresponding moves the bot takes using an A* search algorithm.

Let's go through this process step-by-step. Since we need multiple ships in order to compute an average percentage, we can test this with 10 ships as a general starting point. Although this project's constraints outline that D (the dimension of the ship) should not be less than 30, we decided to make each of the 10 ships as 10 x 10 because although our algorithm would work with 30 x 30 ships, the runtime would be significantly longer, especially since we're repeating the same algorithm for 10 different ships.

For each ship, we would generate the ship itself and create the T configuration array for that ship. We can also use two counters that would help us compute the average: the number of matches between the optimal path and shortest path, and the number of configurations we look at. We can initialize both of these at 0 to start off.

Then for each possible bot and rat position (we kept track of this using a list of open cells in the ship - similar to how we did when computing T), we first check if the bot and the rat are in the same cell for the current configuration. If they are, then we skip it, since the optimal path and shortest path wouldn't result in the bot moving for such cases since it already found the rat. If the bot hasn't found the rat in the current configuration, we calculate the bot's best move to the rat using our optimal search algorithm (which was outlined earlier in this report); remember that the bot's best move is considered to be a single step, not an overall path to the rat, and would be in the form of coordinates that the bot would move to. We then find the shortest path from the bot to the rat using A* search (our A* search algorithm is the exact same search algorithm we used in project 2); the shortest path would be a list of coordinates that the bot would (sequentially) move to. We then extract what the bot's first action would have been using the shortest path by extracting the second set of coordinates in the "shortest path" list; remember that the first set of coordinates in the list would be the bot's starting position (i.e. its current position), not its next position after moving. We then compare if the bot's "optimal action" is the same as its "shortest path" action. If they are, we can increment the match counter by 1 since the optimal path would be the shortest path for that one step. We then increment the total configuration counter by 1 since we've checked if both algorithms match for one single configuration in the T array.

Theoretically, we could repeat this process and make comparisons between each step (for every step) in the optimal path algorithm and the shortest path algorithm, but there is no guarantee that both algorithms would result in the same number of steps. For example, the optimal path for one configuration could have 12 steps (i.e. 12 different coordinates/bot positions), but the shortest path might have 9 steps. For this reason, we'll only look at the first step for both search algorithms for each configuration in T.

After looking through the entire T array and comparing matches between the optimal path and the shortest path, we calculate the percentage using the following formula:

$$Percentage = (\frac{total\ number\ of\ matches}{total\ number\ of\ configurations}) * 100$$

(Note that this formula would only work if there is at least 1 configuration in the T array. Otherwise, the percentage would be 0.)

After doing the testing across multiple randomly generated ship layouts, the bot's optimal action—computed using the value function that accounts for rat movement randomness—matches the first step along the shortest path (A*) to the rat in approximately 45.54% of all reachable configurations after a couple iterations. Of course, this approximation can change as the ship being generated is always random, so results might not be consistent, though this method of comparison is still effective. The percentage of matches between the optimal path algorithm and the shortest path algorithm being 45.54% suggests that while following the shortest path is sometimes optimal, the majority of the time the optimal strategy deviates from it to account for the rat's possible movements, such as intercepting or anticipating where the rat might go. Therefore, simply chasing the rat using the shortest path is not optimal in most cases.

Let's consider how many possible configurations there can be in T if there are N open cells in a ship. What is the relationship between the number of open cells and the size of the configuration array T be affected?

If there are N open cells in a ship, that means that there are N possible positions that the rat and bot can be in, assuming that they can only be in open cells and not blocked cells. So there would be N x N configurations. But this is if we're including configurations where the bot and rat are in the same cell, meaning that the rat has been found by the bot. If we excluded such configurations, we would only have N x (N - 1) configurations, where N - 1 is the number of possible positions since out of all the open cells, we're excluding one that has already been taken by either the bot or the rat, thus ensuring that we don't count configurations where both of them are in the same cell.

Let's try to take our optimal path algorithm and configuration array and run it through some machine learning in order to make better predictions for the number of steps it takes for

the bot to reach the rat. Before making the model however, we need to consider how to represent three things: our input, our output, and the model space itself.

Our machine learning model's input should be a 4-dimensional vector: [bot_x, bot_y, rat_x, rat_y]. This vector essentially represents the positions of the bot and the rat in the ship, similar to the (bot, rat) tuples in our configuration array T. This would allow the model to create a mapping from these positions to the expected number of moves. Our output could be a single scalar value, given by T[bx, by, rx, ry], which represents the expected number of steps for the bot to catch the rat. This essentially becomes a regression problem, where the model must learn to predict outcomes with real values based on the given ship.

What should our model space be?

The model space is the set of all functions that a machine learning model can represent and learn from data. For this specific project, our task is to train a model that can approximate a value function T (using value iteration), which would map a given configuration of the bot and rat to the expected number of steps required for the bot to catch the rat. Since we're dealing with linear regression, a neural network is an optimal choice for a model space.

Neural networks consist of different layers: an input layer, 1 or more hidden layers, and an output layer. In our input layer, we're taking in the 4-dimensional vector that represents our current configuration and mapping it to 64 neurons (outputs). These neurons are the features that we're going to calculate. In our first hidden layer, those 64 neurons would be taken in as input and produce 32 neurons/features as output. Our output layer would then take those 32 neurons and produce 1 single scalar value, represented by T[bx, by, rx, ry].

Notice that in our neural network, we pass along the outputs of previous layers as inputs to future layers. This process of mapping inputs to outputs is known as linear transformation. For every layer that uses linear transformation, an activation function is used to make our output data non-linear. The purpose of making our data non-linear is that it allows our model to catch more complex patterns in our data, making our model learn more efficiently. The only layer that doesn't use an activation function is the output layer, as the data that it takes as input has already been made non-linear by the activation function in the previous layer. In this sense, our neural network follows a "forward flow", meaning it forwards the inputs of previous layers to the next layer, allowing the data to "flow".

Let's go through our neural network's flow step-by-step, assuming that we've initialized our neural network's architecture..

First it would apply the input layer to our input and use an activation function such as ReLU to introduce non-linearity into the model, and produce 64 neurons as output, which are passed to our second layer (the hidden layer). The second layer takes these 64 neurons as input and uses a ReLU activation function to further increase the number of spatial features our model is calculating (thus leading to it looking for more complex patterns in the data). The second layer produces 32 neurons as output and passes them to the output layer as input. The output layer then takes these 32 inputs and produces a continuous scalar value representing the predicted expected number of steps for the bot to catch the rat. Since this is a regression problem, no activation function is applied at the output — the model directly outputs a real number. This overall architecture provides a flexible and expressive function approximator for modeling the T values across many different bot–rat configurations.

Now that our model has been initialized and has a thorough process of taking in data, we need to assess how it can be used to train itself on given data and its overall quality (in terms of performance).

A popular training algorithm used in machine learning that we used for this project is stochastic gradient descent with an Adam optimizer. The Adam optimizer improves on basic gradient descent by adjusting the learning rate for each parameter based on estimates computed by gradients; gradients are derivatives calculated to produce estimates for the number of steps. This helps the model converge faster and produces more accurate results, especially when dealing with large and complex datasets, which fits our task perfectly because we often deal with datasets that have at least 30000 datapoints. Adam is also a good choice for an optimizer because it requires minimal tuning with hyperparameters, meaning that it's very adaptable when we're experimenting with different sample sizes and learning rates.

For our training model, we started off with a learning rate of 0.001 as a starting point, tweaking our learning rate after multiple tests to determine the speed at which our machine should learn in order to optimize its efficiency. The model's efficiency (or quality) can be determined by the loss function, which calculates the difference between the model's predictions and the actual values. For regression tasks such as this, mean squared error (MSE) is most commonly used. During training, we keep track of the average loss per epoch (epoch - every time we look through our sampled data and train it) to monitor the model's learning progress. Once the model has been trained on our data, we evaluate the model by running it with some test data that contains bot-rat configurations that it's never seen before, and check to see how well our model generalizes to these new configurations. For example, a test MSE of around 8.29 implies that the model's predictions are typically within about ±3 moves (since

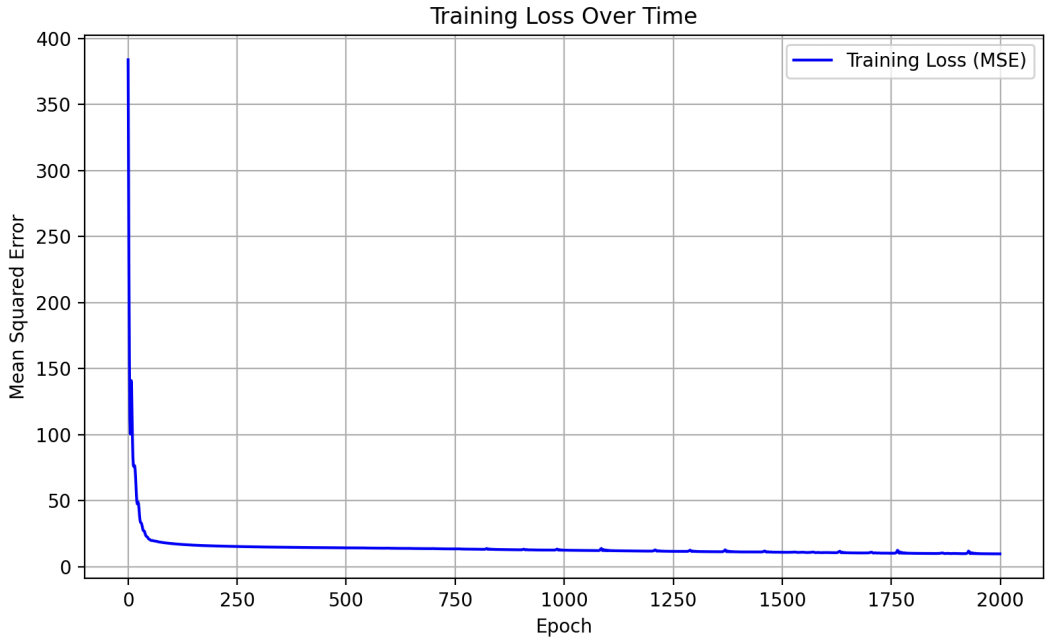
√8.29 ≈ 2.88) of the true value, which provides a useful benchmark for how reliable the model is.

If our model is not performing well enough, we can adjust the hyperparameters like the learning rate, batch size, or even try other optimizers.

What can be an indicator that our model is learning?ˆ

We can deduce that our current model is learning by looking at the mean squared loss, which is calculated for every epoch. Specifically, we can measure how the loss changes over time.

Below is a graph showing how the loss changes over time when training data gathered from the T array of a 20 x 20 ship (30 x 30 ship may increase the runtime of our algorithm).



As you can see, as we train our data more and more (number of epochs increases), the mean squared error decreases exponentially, eventually reaching a plateau where the change in loss is not as significant. This shows that our model is getting more accurate the more it trains with our data to make more accurate predictions, minimizing the loss (the difference between its predicted number of steps and the actual number of steps).

```
Epoch 1996 - Average Loss: 6.5269
Epoch 1997 - Average Loss: 6.5563
Epoch 1998 - Average Loss: 6.5916
Epoch 1999 - Average Loss: 6.6658
Epoch 2000 - Average Loss: 6.6880
Test MSE: 7.8662
```

After running 2000 epochs (training our data 2000 times) for a 20 x 20 ship, our data only had an average loss of 7.8662. While it might not be 0 or a very small decimal close to 0, it is considered to be a good MSE considering that our data is 20 x 20 x 20 x 20 and our sample size is 20000, making our training sample around 36000 - 37000 datapoints. A loss of about 7 for 36000 datapoints is not a significant loss, thus proving our model is effective at making predictions.

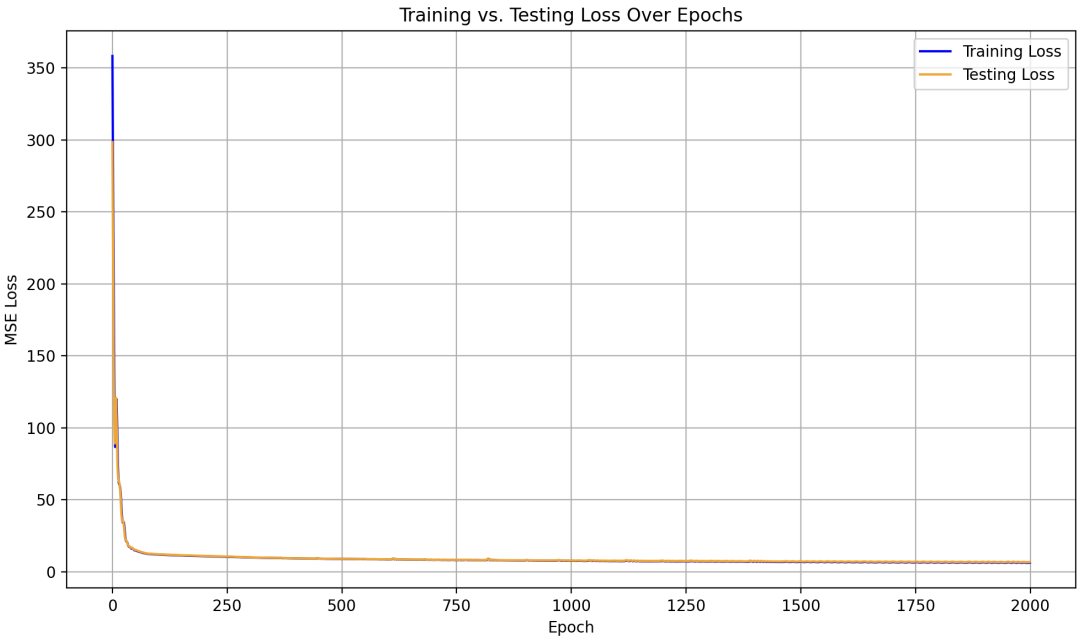
Of course, this MSE varies per ship and per data sample. At times, it might pick a random sample that includes favorable configurations (i.e. configurations that are easy to predict), in which the MSE would be smaller.

```
Epoch 1997 - Average Loss: 5.6620
Epoch 1998 - Average Loss: 5.5577
Epoch 1999 - Average Loss: 5.4312
Epoch 2000 - Average Loss: 5.3422
Test MSE: 5.7518
```

But after doing extensive testing, we can conclude that our MSE generally falls in the range of 4 - 9, proving our model is largely effective at making predictions.ˆ

Of course, we also have to consider if our model is just memorizing the individual data points. We can determine this by comparing both the model's loss with its training data, and its loss with its testing data.

Below is a comparison between the training loss and testing loss for data taken from the T array of a 20 x 20 ship.



As you can see, the testing loss is decreasing along with the training loss, which means that the model is generalizing its data, not memorizing data points in our sample. However, in the beginning the testing loss does seem to deviate from the training loss, but that is to be expected since at that point, the model hasn't been trained enough and so is still making predictions that are not as accurate as its future predictions when trained longer.

After you are able to get a model working on data from one ship, try to build and train a model capable of working across multiple ships. What has to change? Be clear and specific about your approach and your results

Now that we've gotten a model working on data from one ship, let's consider how to modify it so that it can train using data from multiple ships. In other words, we have to augment the model.

In our original model, we use 4 parameters as inputs, which were simply the positions of the bot and rat. We would use these parameters to train our model. This gives decent results when testing on the same ship, but that is not the case when testing our trained model on a different ship. After testing our model on a different ship, we get a MSE (loss) of 72, which is much larger than the MSE when testing our original ship.

```
Generated new ship for testing.
Converged after 31 iterations with average change 0.2826
Final T stats: min=0.00, max=inf
New ship test data size: 12017 examples.
[NEW SHIP OG MODEL] Generalization Test MSE: 72.6116
```

One reason that this large MSE could occur is because the layout of every generated ship is random. For example, we could have a ship where the bot and rat are in certain spots, and specific cells are open or blocked. What if another ship was generated where the bot and rat are in the same spots, but because the number of open and blocked cells are different on this ship, the model might try to follow the same optimal path it used for the previous ship that it trained with, resulting in a higher loss because the model is unfamiliar with the new layout of this ship.

We can mitigate this problem by adding a 5th parameter that would guide the model by telling it how the ship's layout is shaped relative to the positions of the bot and the rat. For example, the 5th parameter could be the average amount of blocked cells in between the bot's and rat's positions. We can calculate the average amount of blocked cells by looking at a smaller rectangular region in which the bot and rat are contained (in the ship) and find the average amount of blocked cells within that region; this entire process can be written as a separate function, and we can create a new model class for our augmented model where everything is the same except the number of parameter inputs is 5 instead of 4.

Upon testing our new augmented model with a new ship, we can see the MSE has decreased significantly, from 72 to around 15 as an average loss. This shows significant improvement with our model, however we can also consider other reasons why this change in loss occurred. One likely reason could be that our previous model often overfitted the data because it was so specific with its parameters; we had to test it with 2000 epochs at a learning rate of 0.008 to get the MSE to a desirable number. But our augmented model is more generalized as it's trained using 5 different ships and creates a large dataset from those ships, allowing us to get better results.

```
[AUGMENTED] Epoch 296 - Avg Loss: 10.0576
[AUGMENTED] Epoch 297 - Avg Loss: 10.1076
[AUGMENTED] Epoch 298 - Avg Loss: 10.0557
[AUGMENTED] Epoch 299 - Avg Loss: 10.5239
[AUGMENTED] Epoch 300 - Avg Loss: 10.1685
Generated new ship for testing.
Converged after 32 iterations with average change 0.2948
Final T stats: min=0.00, max=inf
New ship test data size: 11754 examples.
[NEW SHIP] Generalization Test MSE: 15.6035
(.venv) gilbe@MacBook-Air VScode files %
```